Solution: Regular Expression Matching

Problem Statement

Given a string `text` and regular expression `pattern`, return `true` if string `text` matches with `pattern`. Otherwise, return `false`.

Follow the below rules to match `text` with `pattern`.

- '.' matches any `single character`.
- '*' matches zero or more occurrences of the `preceding element`.

Examples

- **Example 1**:
  - **Input**: text = "abc", pattern = "a.c"
  - **Expected Output**: **true**
  - **Justification**: The '.' in the pattern matches any character, so "a.c" matches "abc".
- **Example 2**:
  - **Input**: text = "aadg", pattern = ".*d."
  - **Expected Output**: **true**
  - **Justification**: The pattern ".*d." matches any string that contains "d" followed by any character, which "aadg" does.
- **Example 3**:
  - **Input**: text = "hello", pattern = "he*o"
  - **Expected Output**: **false**
  - **Justification**: The string 'hello' doesn't match the pattern "he*o" as the pattern referes to strings like "heo", "heeo", "heeeo", etc.

Solution

To solve this problem, we approach it with dynamic programming due to the optimal substructure and overlapping subproblems inherent to pattern matching with wildcards. This approach efficiently breaks down the problem into smaller, manageable subproblems, solving each once and storing their solutions for future reference, thereby avoiding redundant computations. The idea is to use a 2D table where each cell represents the match status between substrings of the text and pattern up to their respective positions. By iteratively filling this table based on the current and previous states, we can determine the match status for the entire string.

This method is effective because it systematically explores all possible matches, including those involving the special characters '.' and '*', ensuring all scenarios are covered. By building the solution from simpler cases, we ensure a thorough examination of the pattern's structure relative to the text, making this approach both comprehensive and scalable.

Step-by-step Algorithm

1. Initialize a 2D boolean array `dp` where `dp[i][j]` represents whether the first `i` characters of the text match the first `j` characters of the pattern.
2. Set `dp[0][0]` to true, as an empty text matches an empty pattern.
3. Fill the first row of `dp`, considering patterns with '*' that can match zero characters.
4. Iterate through each character of the text (i) and pattern (j):
   - If the pattern character is '.', or it matches the current text character, set `dp[i][j]` based on the diagonal previous (`dp[i-1][j-1]`).
   - If the pattern character is '*', check two conditions:
     - Zero occurrences of the character before '*': `dp[i][j-2]`.
     - One or more occurrences: If the character before '*' matches the current text character, use the result from the row above (`dp[i-1][j]`).
5. The final answer is in `dp[text.length()][pattern.length()]`, indicating the match status for the entire strings.

Algorithm Walkthrough

Let's consider the input: text = "aadg", pattern = ".*d."

Initial Matrix Setup

Initial `dp` setup (T for True, F for False):

```
      ""  *  .  d  .
0 ""  T   F   F  F  F
1 a   F
2 a   F
3 d   F
4 g   F
```

Step 1: Fill the First Row

Since ".*" can match any sequence of characters including the empty string, we update dp[0][2] to dp[0][0] as dp[0][2] is `*`.

After updating for ".*", the first row looks like this:

```
      ""  .  *  d  .
0 ""  T   F  T  F  F
1 a   F
2 a   F
3 d   F
4 g   F
```

Step 2: Process Each Character of Text and Pattern

i = 1 (text character = 'a')

- **j = 1 (pattern = '.')**: Since '.' can match any single character, we look at `dp[i-1][j-1]`. `dp[0][0]` is True, so update `dp[1][1]` to True.
- **j = 2 (pattern = '*')**: Here, `*` means zero or more of the preceding element. Since dp[1][0] is True (an empty string matches an empty pattern), and '.' can match any character, dp[1][2] becomes True because '*' allows the '.' to match zero or more characters.
- **j = 3 (pattern = 'd') and j = 4 (pattern = '.')**: We cannot match 'd' or '.' yet because we need the full context of the pattern to match. So, `dp[1][3]` and `dp[1][4]` remain False.

After iteration i=1:

```
      ""  .  *  d  .
0 ""  T   F  T  F  F
1 a   F   T  T  F  F
2 a   F
3 d   F
4 g   F
```

i = 2 (text character = 'a', again)

- **j = 1**: `dp[2][1] = dp[1][0] = False`, as `pattern[j - 1]` is '.'
- **j = 2** ('*'): Given that "*" can match any preceding characters, `dp[2][2]` is True.
- **j = 3 and j = 4**: Similar to the previous row, `dp[2][3]` and `dp[2][4]` remain False due to the lack of a full pattern match.

After iteration i=2:

```
      ""  .  *  d  .
0 ""  T   F  T  F  F
1 a   F   T  T  F  F
2 a   F   F  T  F  F
3 d   F
4 g   F
```

i = 3 (text character = 'd')

- **j = 1**: `dp[3][1] = dp[2][0] = False`, as `pattern[j - 1]` is '.'
- **j = 2** ('*'): Given that "*" can match any preceding characters, `dp[3][2]` is True.
- **j = 3** ('**d**'): Now, 'd' in the text matches 'd' in the pattern. Since `dp[2][2]` is True (matching "aa" with ".*"), `dp[3][3]` becomes True.
- **j = 4** (**final** '.'): We can't match this '.' yet, as it depends on matching the entire string and pattern, so `dp[3][4]` remains False.

After iteration i=3:

```
      ""  .  *  d  .
0 ""  T   F  T  F  F
1 a   F   T  T  F  F
2 a   F   F  T  F  F
3 d   F   F  T  T  F
4 g   F
```

i = 4 (text character = 'g')

- **j = 1**: `dp[4][1] = dp[3][0] = False`, as `pattern[j - 1]` is '.'
- **j = 2** ('*'): "*." *can match "aad", so dp[4][2] is True because* " can cover any number of characters.
- **j = 3** ('**d**'): There's no direct match for 'g' with 'd', so `dp[4][3]` remains False.
- **j = 4** (**final** '.'): Now, this '.' can match 'g', and since dp[3][3] is True (matching "aad" with ".*d"), `dp[4][4]` becomes True, indicating the entire string

"aadg" matches the pattern ".*d.".

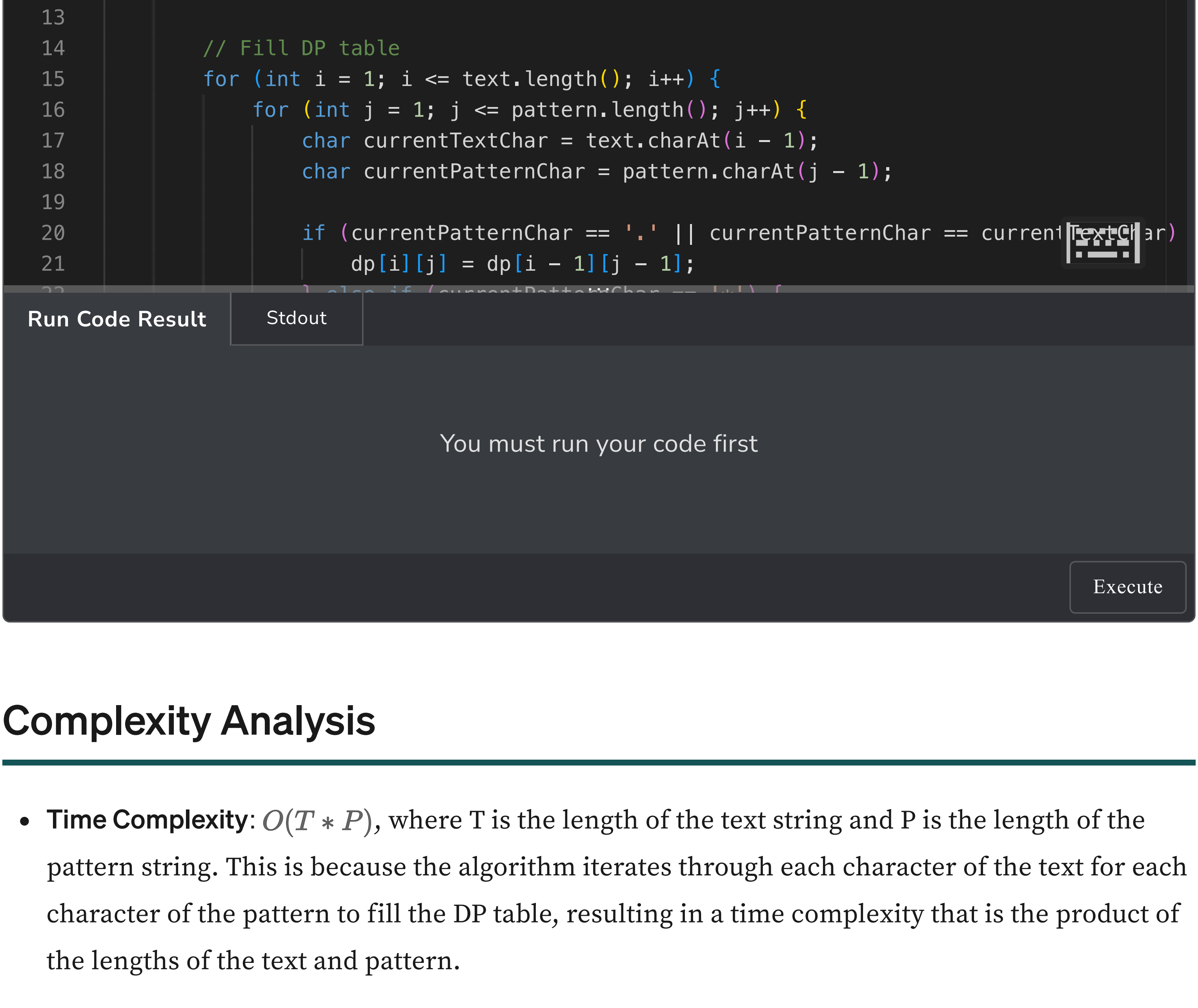
Final matrix after iteration i=4:

```
      ""  .  *  d  .
0 ""  T   F  T  F  F
1 a   F   T  T  F  F
2 a   F   F  T  F  F
3 d   F   F  T  T  F
4 g   F   F  T  F  T
```

Conclusion

The value at `dp[text.length()][pattern.length()]`, which is `dp[4][4]`, tells us if the text "aadg" matches the pattern ".*d.". In this case, `dp[4][4]` = True, showing that "aadg" indeed matches ".*d.", completing the step-by-step walkthrough accurately.

Code



```java
public class Solution {
    public boolean isMatch(String text, String pattern) {
        // Initialize DP table
        boolean[][] dp = new boolean[text.length() + 1][pattern.length() + 1];
        dp[0][0] = true; // Empty text matches empty pattern

        // Handle patterns like a*, a*b*, a*b*c* etc.
        for (int j = 2; j <= pattern.length(); j += 2) {
            if (pattern.charAt(j - 1) == '*') {
                dp[0][j] = dp[0][j - 2];
            }
        }

        // Fill DP table
        for (int i = 1; i <= text.length(); i++) {
            for (int j = 1; j <= pattern.length(); j++) {
                char currentTextChar = text.charAt(i - 1);
                char currentPatternChar = pattern.charAt(j - 1);

                if (currentPatternChar == '.' || currentPatternChar == currentTextChar)
                    dp[i][j] = dp[i - 1][j - 1];
```

You must run your code first

Complexity Analysis

- **Time Complexity**: $O(T * P)$, where T is the length of the text string and P is the length of the pattern string. This is because the algorithm iterates through each character of the text for each character of the pattern to fill the DP table, resulting in a time complexity that is the product of the lengths of the text and pattern.
- **Space Complexity**: $O(T * P)$, due to the use of a 2D DP table that has a size proportional to the product of the lengths of the text and pattern.

← Previous
Regular Expression Matching (Hard)

Next →
First Missing Positive (Hard)

☐ Mark as Completed